

Savannah Hamley

Karthika Subramani

CSCI4760

March 19, 2026

Assignment 2: Part 1

Docker Compose file and its Output:

The docker-compose.yml file is created to define and run the required containers for this assignment. This file creates a client and a server container that communicate with each other through the Docker bridge network. This maps a host port to the server container port, allowing the host to connect to the server application running inside of the container. The script also created folders for the client and server. These folders are mounted into the containers using Docker volumes, which allows changes to the Python files on the host machine to reflect inside the containers. This setup allows the user to run and manage the client and server environments efficiently.

Server and Client Code:

Server:

```
import os
import socket
import sys
from typing import Tuple

# Retrieve configurations from Docker environment variables
USERNAME = os.getenv("STUDENT_USERNAME")
HOST = "0.0.0.0" # Listen on all interfaces within the container
PORT = int(os.getenv("APP_PORT"))

def create_listen_socket(host: str, port: int) -> socket.socket:
    """Create, bind, and listen on a TCP socket."""
    # Initialize a TCP socket using IPv4 (AF_INET) and Streaming (SOCK_STREAM)
```

```

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Allow the socket to be reused immediately after the server stops
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# Bind the socket to the defined host and port
s.bind((host, port))

# Enable the server to accept connections, with a backlog of 5
s.listen(5)
return s

def handle_client(conn: socket.socket, addr: Tuple[str, int]) -> None:
    """Echo loop for a single client."""
    print(f"[server:{USERNAME}] connection from {addr}")
    try:
        while True:
            # Receive up to 4096 bytes of data from the client
            data = conn.recv(4096)

            # If recv returns empty bytes, the client has disconnected
            if not data:
                break

            # Log the received message with the student username
            print(f"[server:{USERNAME}] received: {data.decode()!r}")

            # Echo the received data back to the client
            conn.sendall(data)
    finally:
        # Ensure the connection is closed when the loop breaks
        conn.close()

def main() -> None:
    try:
        lsock = create_listen_socket(HOST, PORT)
    except Exception as e:
        print(f"[server:{USERNAME}] failed to create listen socket: {e}",
file=sys.stderr)
        sys.exit(1)

    print(f"[server:{USERNAME}] listening on {HOST}:{PORT}")
    while True:
        try:
            # Wait for a new client connection
            conn, addr = lsock.accept()
            # Pass the connection to the handler function
            handle_client(conn, addr)

```

```

        except KeyboardInterrupt:
            print("\n[server] shutting down.")
            break
        except Exception as e:
            print(f"[server:{USERNAME}] error: {e}", file=sys.stderr)
    lsock.close()

if __name__ == "__main__":
    main()

```

Client:

```

import os
import socket
import sys
import time

# Retrieve credentials and connection details from Docker environment
USERNAME = os.getenv("STUDENT_USERNAME") or "Unknown"
SERVER_HOST = os.getenv("SERVER_HOST")
SERVER_PORT = int(os.getenv("SERVER_PORT"))
CONNECT_TIMEOUT = 5

def connect_once(host: str, port: int) -> socket.socket:
    """Create a TCP connection to the server."""
    # create_connection handles both the socket creation and the 3-way
    handshake
    return socket.create_connection((host, port), timeout=CONNECT_TIMEOUT)

def main() -> None:
    print(f"[client:{USERNAME}] target -> {SERVER_HOST}:{SERVER_PORT}")
    try:
        sock = connect_once(SERVER_HOST, SERVER_PORT)
    except Exception as e:
        print(f"[client:{USERNAME}] connection failed: {e}", file=sys.stderr)
        sys.exit(1)

    print(f"[client:{USERNAME}] connected. Type messages; Ctrl+C to quit.")
    try:
        while True:
            # Take interactive user input from the terminal
            message = input("> ")

            # Prepend username so the traffic is identifiable in Wireshark
            full_message = f"{USERNAME}: {message}"

```

```

        # Convert string to bytes before sending over the network
        sock.sendall(full_message.encode())

        # Receive the echoed response from the server
        data = sock.recv(4096)

        print(f"[echo] {data.decode()}")
        time.sleep(0.1)

    except KeyboardInterrupt:
        print("\n[client] exiting.")
    finally:
        # Close the socket to trigger the FIN/ACK sequence in Wireshark
        sock.close()

if __name__ == "__main__":
    main()

```

Challenges Faced:

A challenge I faced was determining which network interface to use when capturing traffic in Wireshark. Initially, no packets appeared when I tried to monitor the data. I then realized that the traffic was routed through the loopback (lo0) interface. After I changed the interface and applied the appropriate TCP filter, I was able to capture the handshake and message between the client and server.

Steps for Network Sniffing:

1. Start Wireshark and select the loopback (lo0) interface to capture traffic.
2. Apply a TCP port filter. I applied `tcp.port == 25297` to ensure only TCP packets associated with the server port were displayed.
3. Start capturing packets in Wireshark, then run the server program inside the server container using the terminal commands:

```
docker compose exec server bash
```

```
python server.py
```

In another terminal, run the client program inside the client container using the terminal commands:

```
docker compose exec client bash
```

```
python client.py
```

4. In Wireshark, identify the TCP Handshake. This shows *SYN*, *SYN/ACK*, and *ACK* flags in the three-way handshake attempt between client and server.

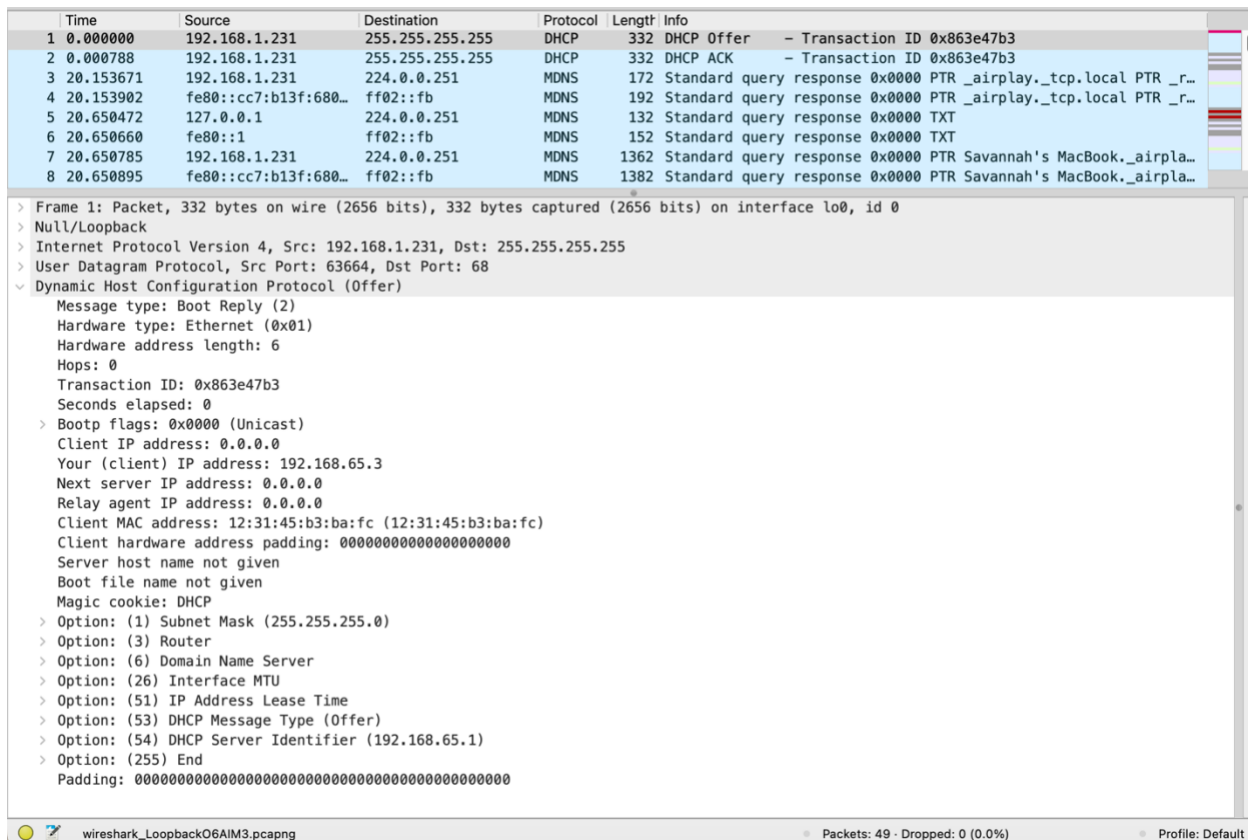


Fig. 1. TCP three-way handshake

5. Identify the client-to-server data packet. After the program was executed, Wireshark captured multiple TCP packets exchanged between the client and server. I located a packet where the client sent data to the server. This packet was marked with *PSH*, *ACK* flags.

No.	Time	Source	Destination	Protocol	Length	Info
2	0.654454	:::1	:::1	TCP	82	63916 → 25297 [PSH, ACK] Seq=1 Ack=1 Win=6372 Len=6 TSval=16576
3	0.654563	:::1	:::1	TCP	76	25297 → 63916 [ACK] Seq=1 Ack=7 Win=6372 Len=0 TSval=16576
4	2.565486	:::1	:::1	TCP	79	63916 → 25297 [PSH, ACK] Seq=7 Ack=1 Win=6372 Len=3 TSval=16576
5	2.565567	:::1	:::1	TCP	76	25297 → 63916 [ACK] Seq=1 Ack=10 Win=6372 Len=0 TSval=16576
6	4.774760	:::1	:::1	TCP	81	63916 → 25297 [PSH, ACK] Seq=10 Ack=1 Win=6372 Len=5 TSval=16576
7	4.774837	:::1	:::1	TCP	76	25297 → 63916 [ACK] Seq=1 Ack=15 Win=6372 Len=0 TSval=16576
8	19.774873	:::1	:::1	TCP	64	[TCP Keep-Alive] 25297 → 63916 [ACK] Seq=0 Ack=15 Win=6372 Len=0
9	19.774980	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 63916 → 25297 [ACK] Seq=15 Ack=1 Win=6372 Len=0

> Frame 2: Packet, 82 bytes on wire (656 bits), 82 bytes captured (656 bits) on interface lo0, id 0

> Null/Loopback

> Internet Protocol Version 6, Src: ::1, Dst: ::1

▼ Transmission Control Protocol, Src Port: 63916, Dst Port: 25297, Seq: 1, Ack: 1, Len: 6

Source Port: 63916

Destination Port: 25297

[Stream index: 0]

[Stream Packet Number: 1]

> [Conversation completeness: Incomplete (12)]

[TCP Segment Len: 6]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 2665549989

[Next Sequence Number: 7 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment number (raw): 1699112000

1000 = Header Length: 32 bytes (8)

> Flags: 0x018 (PSH, ACK)

Window: 6372

[Calculated window size: 6372]

[Window size scaling factor: -1 (unknown)]

Checksum: 0x002e [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

> Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps

> [Timestamps]

[Client Contiguous Streams: 0]

[Server Contiguous Streams: 1]

TCP payload (6 bytes)

▼ Data (6 bytes)

Data: 68656c6c6f0a

[Length: 6]

wireshark_LoopbackZAOBM3.pcapng Packets: 13 - Displayed: 10 (76.9%) - Dropped: 0 (0.0%) Profile: Default

Fig. 2. Client sending TCP data packet

- Identify the server acknowledgment packet. I observed a packet where the server acknowledged receiving the client's message. This packet contained the *ACK* flag. This indicates that the server successfully received the data sent by the client.

No.	Time	Source	Destination	Protocol	Length	Info
2	0.654454	:::1	:::1	TCP	82	63916 → 25297 [PSH, ACK] Seq=1 Ack=1 Win=6372 Len=6 TSval=16572
3	0.654563	:::1	:::1	TCP	76	25297 → 63916 [ACK] Seq=1 Ack=7 Win=6372 Len=0 TSval=16572
4	2.565486	:::1	:::1	TCP	79	63916 → 25297 [PSH, ACK] Seq=7 Ack=1 Win=6372 Len=3 TSval=16572
5	2.565567	:::1	:::1	TCP	76	25297 → 63916 [ACK] Seq=1 Ack=10 Win=6372 Len=0 TSval=16572
6	4.774760	:::1	:::1	TCP	81	63916 → 25297 [PSH, ACK] Seq=10 Ack=1 Win=6372 Len=5 TSval=16572
7	4.774837	:::1	:::1	TCP	76	25297 → 63916 [ACK] Seq=1 Ack=15 Win=6372 Len=0 TSval=16572
8	19.774873	:::1	:::1	TCP	64	[TCP Keep-Alive] 25297 → 63916 [ACK] Seq=0 Ack=15 Win=6372 Len=0 TSval=16572
9	19.774980	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 63916 → 25297 [ACK] Seq=15 Ack=1 Win=6372 Len=0 TSval=16572


```

> Frame 3: Packet, 76 bytes on wire (608 bits), 76 bytes captured (608 bits) on interface lo0, id 0
> Null/Loopback
> Internet Protocol Version 6, Src: ::1, Dst: ::1
< Transmission Control Protocol, Src Port: 25297, Dst Port: 63916, Seq: 1, Ack: 7, Len: 0
  Source Port: 25297
  Destination Port: 63916
  [Stream index: 0]
  [Stream Packet Number: 2]
  > [Conversation completeness: Incomplete (12)]
  [TCP Segment Len: 0]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 1699112000
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 7 (relative ack number)
  Acknowledgment number (raw): 2665549995
  1000 .... = Header Length: 32 bytes (8)
  > Flags: 0x010 (ACK)
  Window: 6372
  [Calculated window size: 6372]
  [Window size scaling factor: -1 (unknown)]
  Checksum: 0x0028 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  > Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
  > [Timestamps]
  > [SEQ/ACK analysis]
  [Client Contiguous Streams: 1]
  [Server Contiguous Streams: 0]

```

wireshark_LoopbackZAOBM3.pcapng Packets: 13 · Displayed: 10 (76.9%) · Dropped: 0 (0.0%) · Profile: Default

Fig. 3. Server acknowledgment packet

- Observe additional TCP communication between the client and server. Wireshark also captured additional packets as additional messages were transmitted and acknowledged between both programs.

No.	Time	Source	Destination	Protocol	Length	Info
2	0.654454	:::1	:::1	TCP	82	63916 → 25297 [PSH, ACK] Seq=1 Ack=1 Win=6372 Len=6 TSval=1228423931 TSecr=1657743518
3	0.654563	:::1	:::1	TCP	76	25297 → 63916 [ACK] Seq=1 Ack=7 Win=0 Len=0 TSval=1657811277 TSecr=1228423931
4	2.565486	:::1	:::1	TCP	79	63916 → 25297 [PSH, ACK] Seq=7 Ack=1 Win=6372 Len=3 TSval=1228425842 TSecr=1657811277
5	2.565567	:::1	:::1	TCP	76	25297 → 63916 [ACK] Seq=1 Ack=10 Win=6372 Len=0 TSval=1657813188 TSecr=1228425842
6	4.774760	:::1	:::1	TCP	81	63916 → 25297 [PSH, ACK] Seq=10 Ack=1 Win=6372 Len=5 TSval=1228428051 TSecr=1657813188
7	4.774837	:::1	:::1	TCP	76	25297 → 63916 [ACK] Seq=1 Ack=15 Win=6372 Len=0 TSval=1657815397 TSecr=1228428051
8	19.774873	:::1	:::1	TCP	64	[TCP Keep-Alive] 25297 → 63916 [ACK] Seq=0 Ack=15 Win=6371 Len=0
9	19.774980	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 63916 → 25297 [ACK] Seq=15 Ack=1 Win=6372 Len=0 TSval=1228443051 TSecr=1657815397
12	34.774819	:::1	:::1	TCP	64	[TCP Keep-Alive] 25297 → 63916 [ACK] Seq=0 Ack=15 Win=6371 Len=0
13	34.774896	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 63916 → 25297 [ACK] Seq=15 Ack=1 Win=6372 Len=0 TSval=1228458051 TSecr=1657815397


```

> Frame 4: Packet, 79 bytes on wire (632 bits), 79 bytes captured (632 bits) on interface lo0, id 0
> Null/Loopback
> Internet Protocol Version 6, Src: ::1, Dst: ::1
> Transmission Control Protocol, Src Port: 63916, Dst Port: 25297, Seq: 7, Ack: 1, Len: 3
  Source Port: 63916
  Destination Port: 25297
  [Stream index: 0]
  [Stream Packet Number: 3]
  [Conversation completeness: Incomplete (12)]
  [TCP Segment Len: 3]
  Sequence Number: 7 (relative sequence number)
  Sequence Number (raw): 2665549995
  [Next Sequence Number: 10 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 1699112000
  1000 .... = Header Length: 32 bytes (8)
  Flags: 0x018 (PSH, ACK)
  Window: 6372
  [Calculated window size: 6372]
  [Window size scaling factor: -1 (unknown)]
  Checksum: 0x002b [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
  [Timestamps]
  [SEQ/ACK analysis]
  [Client Contiguous Streams: 0]
  [Server Contiguous Streams: 1]
  TCP payload (3 bytes)
  Data (3 bytes)
  Data: 68690a
  [Length: 3]

```

wireshark_LoopbackZA0BM3.pcapng Packets: 13 · Displayed: 10 (76.9%) · Dropped: 0 (0.0%) Profile: Default

Fig. 4. Additional TCP communication

Observations:

The following observations were collected from captured TCP packets in Wireshark.

These values represent the communication between the client and server containers as they communicate. The TCP sequence number represents the position of the first byte of data in the segment, while the acknowledgment number confirms receipt of bytes from the other host. (The Client and Server IP addresses were determined by inspecting the Docker bridge network.)

Item	Value
Client IP	172.18.0.2
Server IP	172.18.0.3
Client Port	63916
Server Port	25297
Sequence Number Example	1
Acknowledgment Number Example	1

Assignment 2: Part 2

Modified/Added Code:**Server:**

```

import os
import socket
import sys
import threading # 1. Added: Import the threading module to handle concurrent
clients
from typing import Tuple

# Retrieve configurations from Docker environment variables
USERNAME = os.getenv("STUDENT_USERNAME")
HOST = "0.0.0.0"
PORT = int(os.getenv("APP_PORT"))

def create_listen_socket(host: str, port: int) -> socket.socket:
    """Create, bind, and listen on a TCP socket."""
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((host, port))
    s.listen(5)
    return s

def handle_client(conn: socket.socket, addr: Tuple[str, int]) -> None:
    """Echo loop for a single client - this now runs in its own thread."""
    print(f"[server:{USERNAME}] connection from {addr}")
    try:
        while True:
            data = conn.recv(4096)
            if not data:
                break
            print(f"[server:{USERNAME}] received from {addr}:
{data.decode()!r}")
            conn.sendall(data)
    finally:
        conn.close()
        print(f"[server:{USERNAME}] connection from {addr} closed")

def main() -> None:
    try:
        lsock = create_listen_socket(HOST, PORT)
    except Exception as e:

```

```

        print(f"[server:{USERNAME}] failed to create listen socket: {e}",
file=sys.stderr)
        sys.exit(1)

    print(f"[server:{USERNAME}] listening on {HOST}:{PORT}")

    while True:
        try:
            # Wait for a new client connection
            conn, addr = lsock.accept()

            # 2. Changed: Instead of calling handle_client directly (which
blocks),
            # we create a new thread for this specific connection.
            client_thread = threading.Thread(target=handle_client, args=(conn,
addr))

            # 3. Added: Mark as daemon so threads close automatically when the
server stops
            client_thread.daemon = True

            # 4. Added: Start the thread
            client_thread.start()

            # Log active threads to prove concurrency
            print(f"[server:{USERNAME}] Active client threads:
{threading.active_count() - 1}")

        except KeyboardInterrupt:
            print("\n[server] shutting down.")
            break
        except Exception as e:
            print(f"[server:{USERNAME}] error: {e}", file=sys.stderr)
            lsock.close()

if __name__ == "__main__":
    main()

```

Client:

No modifications were required for client.py since the server is managed through multithreading. The connection process remains identical regardless.

Updated Docker Compose file:

The docker-compose.yml file did not need to be changed. This allows for the necessary scaling required for testing. The existing client service definition and multiple client instances can be instantiated via the command line.

```
version: "3.9"
services:
  server:
    image: python:3.11-slim
    container_name: csci-server-hamley
    working_dir: /app
    volumes:
      - ./server-hamley:/app
    environment:
      - STUDENT_USERNAME=hamley
      - APP_PORT=8529
    command: bash
    tty: true
    stdin_open: true
    networks: [ csci-net-hamley ]
    ports:
      - "25297:8529"

  client:
    image: python:3.11-slim
    container_name: csci-client-hamley
    working_dir: /app
    volumes:
      - ./client-hamley:/app
    environment:
      - STUDENT_USERNAME=hamley
      - SERVER_HOST=csci-server-hamley
      - SERVER_PORT=8529
    command: bash
    tty: true
    stdin_open: true
    networks: [ csci-net-hamley ]

networks:
  csci-net-hamley:
    driver: bridge
```

README:

This program utilizes a container where `PROXY_PORT` (mapped to 25297) serves as the host entry point, allowing external traffic from clients to reach the server container. The `SERVER_PORT` (8529) is the internal port on which the server application listens.

Running and Scaling Instructions:

1. To set up the environment, execute *`docker-compose up -d`* to initialize the bridge network and containers.
2. To initialize the server, execute *`docker exec -it csci-server-hamley python3 server.py`* to start the multi-threaded server process.
3. To add multiple users, open separate terminal windows and execute *`docker exec -it csci-client-hamley python3 client.py`* in each. This allows multiple clients to establish independent TCP sessions on one central server.

Testing:

1. In my terminal at my assignment location, I started the server by running the commands:

```
docker-compose up -d
```

```
docker exec -it csci-server-hamley python3 server.py
```

2. Then in 3 separate terminal windows, I ran the clients using the command:

```
docker exec -it csci-client-hamley python3 client.py
```

The figure displays four terminal windows arranged in a 2x2 grid, illustrating the setup and operation of a multi-threaded server and its clients using Docker.

- Top-Left Window:** Shows the removal of existing containers and the creation of a new network. The command `docker-compose up -d` is executed, resulting in the creation of the `assignment2_csci-net-hamley` network and the starting of the `csci-server-hamley` and `csci-client-hamley` containers. The server then runs `python3 server.py`, which starts listening on `0.0.0.0:8529` and shows incoming connections from `172.18.0.2` and `172.18.0.2`.
- Top-Right Window:** Shows a client terminal running `python3 client.py`. It connects to the server at `csci-server-hamley:8529` and displays the message: `[client:hamley] connected. Type messages; Ctrl+C to quit.`
- Bottom-Left Window:** Shows another client terminal running `python3 client.py`. It also connects to the server at `csci-server-hamley:8529` and displays the same connection message.
- Bottom-Right Window:** Shows a third client terminal running `python3 client.py`. It connects to the server at `csci-server-hamley:8529` and displays the same connection message.

Fig. 1. Multi-threaded servers showing connections from three distinct clients

3. I then sent a message from each client terminal and ensured it was received by the server.

The figure consists of four terminal screenshots arranged in a 2x2 grid, showing the setup and operation of a Docker Compose environment for communication between multiple clients and a server.

Top Left Terminal: Shows the execution of `docker-compose up -d`. It lists the containers `csci-server-hamley` and `csci-client-hamley` as removed, and the network `assignment2_csci-net-hamley` as removed. It then shows the network being created and the containers being started.

Top Right Terminal: Shows the execution of `python3 client.py`. It displays the target IP address `csci-server-hamley:8529` and the connection status. The client sends the message `data`, and the server responds with `hamley: data`.

Bottom Left Terminal: Shows the execution of `python3 client.py`. It displays the target IP address `csci-server-hamley:8529` and the connection status. The client sends the message `hello`, and the server responds with `hamley: hello`.

Bottom Right Terminal: Shows the execution of `python3 client.py`. It displays the target IP address `csci-server-hamley:8529` and the connection status. The client sends the message `test`, and the server responds with `hamley: test`.

Fig. 2. Docker Compose enabling communication between multiple clients